Benchmarking Agents for Proving Theorems in Quantum Algorithms and Quantum Information

Lei Zhang[*1,2], Yusheng Zhao[†1,2], Yimeng Cao[2], Ranyiliu Chen[3], Mingrui Jing[‡1,2], Jizhe Lai[2], Ziao Tang[2], Jingu Xie[2], Hongshun Yao[1,2], Xuanqiang Zhao[§1,4], Guocheng Zhen[2], Chengkai Zhu[¶1], and Xin Wang[‖2]

[1]QudeLeap Research, Shanghai 200030, China
[2]The Hong Kong University of Science and Technology (Guangzhou), Guangdong 511453, China
[3]Quantum Science Center of Guangdong-Hong Kong-Macao Greater Bay Area, Shenzhen 518045, China
[4]The University of Hong Kong, Pokfulam Road, Hong Kong

Lean-QuantumAlg-Bench | Lean-QIT-Bench

arXiv:2607.21533v1 [quant-ph] 23 Jul 2026

Abstract

Formal verification is becoming increasingly practical for quantum computing, yet the ability of AI agents to construct machine-checkable proofs in this domain remains unmeasured. We introduce **Lean-QuantumAlg-Bench** and **Lean-QIT-Bench**, two Lean 4 benchmarks containing 36 and 40 theorem-completion tasks for quantum algorithms and quantum information theory, respectively. Every task compiles in a fixed environment and is evaluated by deterministic proof checking and targeted semantic review, with difficulty weights assigned before model execution. We evaluate four models—GPT-5.5, Kimi K3, DeepSeek V4-Pro, and MiniMax M3—within a common theorem-proving framework under two settings: a task-only baseline and library-augmented deduction (LAD), which additionally provides access to a verified domain library. The highest difficulty-weighted scores are 60.4 out of 100 on the quantum-algorithm benchmark and 59.6 out of 100 on the quantum-information benchmark. LAD improves both score and completion rate in all eight model–benchmark comparisons, with gains of up to 15.9 points, providing evidence that verified libraries can strengthen domain-specific proof agents. The results reveal recurring weaknesses of agentic proving in areas such as quantum simulation, quantum learning, quantum information measures, and entanglement theory. Monetary and wall-clock costs per score point also vary substantially across models, highlighting important capability–efficiency trade-offs. We expect these benchmarks to establish a reproducible baseline for developing more capable and reliable proof agents, and to pave the way toward self-evolving AI scientists for advancing quantum information science.

1 Introduction

Quantum computing combines mathematically precise claims with long chains of domain-specific reasoning. Shor's factoring algorithm [1], for example, reduces factoring to finding the multiplicative order of an element modulo N; a quantum subroutine recovers this order from the period of modular exponentiation. In quantum information theory, the data-processing inequality [2, 3] states that quantum relative entropy cannot increase under a quantum channel. Both results connect abstract hypotheses, typed quantum objects, and operational conclusions. A machine-checkable development can make every link in such a chain available for verification, including assumptions that a conventional derivation may leave implicit.

Formalizing these arguments is demanding for reasons that are intrinsic to the subject. Quantum states and operators carry finite-dimensional and often type-level structure; circuit calculations must connect syntactic transformations with linear-algebraic semantics; and information-theoretic inequalities depend on domains, support conditions, and positivity hypotheses. Textbook notation also suppresses coercions, basis choices, tensor-factor ordering, and index conventions. In Lean, each of these choices becomes part of the theorem interface or its imported environment. A useful domain benchmark should therefore test more than isolated

* Equal contribution. leizhang116.4@gmail.com
† Equal contribution. yushengzhao2020@outlook.com
‡ mingruij0031@gmail.com
§ zhaoxuanqiang7@gmail.com
¶ zhuchengkai7@gmail.com
‖ felixxinwang@hkust-gz.edu.cn

algebra: it should reveal whether a prover can navigate the interfaces through which the mathematics is actually represented.

Formal verification has already reached substantial quantum results, including certified quantum computation in Isabelle/HOL [4], an end-to-end implementation of Shor factorization [5], and a Lean formalization of the generalized quantum Stein lemma [6]. Recent preprints and libraries further develop Lean interfaces for quantum information [7, 8], algorithms [9, 10], error correction [11], and optimization problems [12]. These developments show that proof assistants can support deep quantum mathematics. Meanwhile, formal theorem proving in Lean is increasingly organized as an agentic loop combining retrieval, decomposition, iterative repair, and verifier feedback. Representative systems include COPRA [13], AlphaProof [14], DeepSeek-Prover [15, 16], Numina-Lean-Agent [17], A Minimal Agent [18], and Goedel-Architect [19]. For research-level formalization, Archon uses LeanSearch for task decomposition, iterative refinement, and proof synthesis [20], with LeanSearch v2 extending this line toward global premise retrieval [21].

Formal theorem-proving benchmarks have advanced in parallel. Competition benchmarks such as miniF2F [22] and PutnamBench [23] test whole-proof generation on standardized problems. LeanDojo [24] makes library retrieval an explicit part of proving, while miniCTX [25] evaluates prover use of long project contexts. Ref. [26] documents informal–formal fidelity failures, while Ref. [27] shows that statement defects and evaluation-harness failures can distort benchmark scores. Together, these efforts provide strong methods for evaluating proof search, context use, and benchmark quality. We build on these methods to coordinate Lean evaluation across quantum algorithms and quantum information.

A coordinated benchmark for these domains should satisfy several requirements simultaneously. It should cover domain objects and proof patterns beyond the scope of generic mathematical suites and present the same theorem-completion task format under controlled evaluation conditions. Its construction claims must also match the evidence that exists: Lean compilation and deterministic checks apply to every task, whereas manual comparison with the mathematical source is a targeted safeguard for selected or high-risk tasks. This distinction limits score interpretation to agent performance under the stated checks.

In this work, we introduce **Lean-QuantumAlg-Bench** (QAlg-Bench) and **Lean-QIT-Bench** (QIT-Bench), two coordinated Lean 4 benchmark suites containing 36 and 40 theorem-completion tasks for quantum algorithms and quantum information theory, respectively. Their 76 tasks are organized into six fields: three for quantum algorithms and three for quantum information. Both suites use a common theorem-completion contract and the same deterministic proof checks, while retaining domain-specific definitions and statements. We evaluate four models within a common theorem-proving framework under two settings: a task-only baseline and library-augmented deduction (LAD), which additionally provides access to a verified domain library for consultation alongside the shared theorem-completion task format.

The evaluated tasks provide no hints. This design allows us to examine how difficulty-weighted verification scores vary across models, fields, and library-access conditions, as well as the monetary and full-suite wall-clock costs per score point associated with these outcomes. These controlled single-run measurements establish a reproducible baseline for developing more capable and reliable proof agents, and pave the way toward self-evolving AI scientists for advancing quantum information science.

2 Background and related work

Lean [28, 29] is an interactive theorem prover and a functional programming language based on dependent type theory. A declaration is accepted only after elaboration produces a proof term that the trusted kernel checks against its type. Elaborators, tactics, and automation may search for that term, but they do not enlarge the kernel's notion of correctness. For an agent, the operative task is therefore defined by a theorem statement and its imported environment: proposed terms or tactics transform an evolving proof state, and success means that the resulting declaration compiles in the fixed Lean environment. The Lean Mathematical Library (MathLib) [30] supplies much of the reusable mathematical interface on which such tasks depend. Proof generation, premise selection, retrieval, and repository interaction address complementary parts of the task.

Quantum formalization adds a distinctive layer of typed structure to this interface. States and effects are represented through matrices or operators; measurements and channels impose positivity and normalization conditions; circuit syntax must be related to denotational semantics; and entropy, distinguishability, and coding statements introduce additional analytic hypotheses. Algorithmic developments further require finite groups, polynomials, phase transformations, or simulation primitives. Existing formal work demonstrates several parts of this range, from certified circuit semantics and Shor factorization to quantum-information theorems [4–6]. Recent Lean developments include Lean-Quantum, an infrastructure aimed at AI-assisted formalization of quantum information [7]; the reusable finite-dimensional quantum-information interfaces and coding-theorem endpoints of Lean-QIT [8]; and a formalization of the Shor algorithm family spanning order

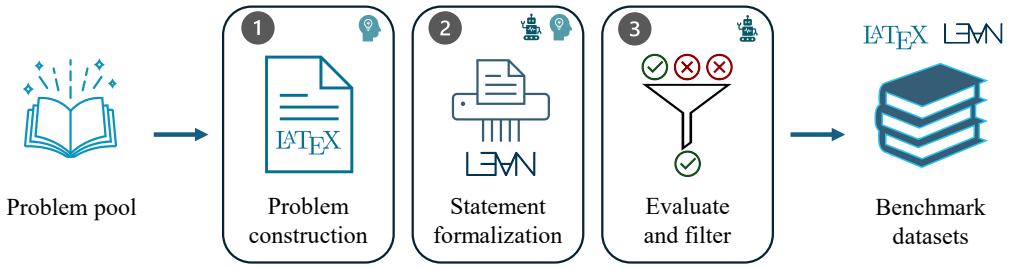

Figure 1: Benchmark construction workflow. Candidate problems are selected and written, translated into Lean, and checked before they enter the QAlg-Bench and QIT-Bench suites.

finding and reversible modular and elliptic-curve arithmetic [9]. The resulting library interfaces determine the actual proof obligations: two informal formulas that look identical may elaborate differently because their types, tensor conventions, or imported definitions differ.

Several early formal-reasoning benchmarks standardized task statements and criteria for proof acceptance. miniF2F [22] translated olympiad-style problems across proof assistants, while PutnamBench [23] extended competition evaluation to university-level Putnam problems and multiple formal systems. MLFMF [31] instead derived Lean and Agda datasets for formalization recommendation from existing libraries. Lean Workbook [32] addressed data scarcity through a large generated-and-filtered collection of informal/formal pairs. These resources differ in whether the unit is a hand-formalized problem, a library declaration, or a generated pair, but each uses fixed formal targets and machine-checkable acceptance.

Later work highlights two limitations of statement-only evaluation that are relevant here: the role of context and the validity of the target itself. LeanDojo [24] combines programmatic repository interaction, fine-grained premise annotations, and retrieval-augmented proving. miniCTX [25] evaluates theorem proving over long project contexts with in-file dependencies. RLMEval [33] selects research-level theorems from Lean blueprint projects, while Ineq-Comp [34] probes robustness under transformed and composed inequalities. Ref. [26] reports that formally accepted statements can still diverge from the intended informal problems. In neighboring domain-specific evaluation, QCircuitBench [35] tests quantum-algorithm code design with automatic validation. Ref. [36] extends Lean evaluation to computational and applied mathematics with explicit attention to dependencies and semantic review. Ref. [27] documents how statement defects and evaluation harnesses can distort formal-proving results.

These developments motivate the combination adopted here. Competition and research benchmarks supply standardized formal targets; repository-level work shows why imported context and library access must be controlled; and quality audits show why kernel acceptance cannot by itself certify the intended meaning of a benchmark statement. Our design coordinates these strands across quantum algorithms and quantum information, partitions their mathematical scope into interpretable fields, and evaluates a fixed task format under baseline and LAD access. QAlg-Bench and QIT-Bench use shared deterministic checks and targeted semantic safeguards. The next section explains their construction; Section 4 reports their descriptive evaluation, and Sections 5 and 6 then describe their field structure and representative tasks.

3 Benchmark construction and validation

3.1 Task design and benchmark construction

In this section, we construct QAlg-Bench and QIT-Bench following the workflow shown in Figure 1. Researchers first convert source problems into theorem-sized tasks. Agents assist in translating these statements into Lean, while researchers retain responsibility for the mathematical choices. Automated checks then compile and screen each task before it is included in one of the two benchmark suites.

The problem pool covers topics treated in established literature on quantum information theory and quantum algorithms. The quantum-information tasks include material treated in standard and specialist texts of the field [3, 37, 38], together with foundational results in quantum coding, channel capacity, entropy, and entanglement [39–45]. The quantum-algorithm tasks cover canonical algorithms and simulation techniques represented by Refs. [1, 46–48].

Candidates are selected for scientific relevance, theorem-sized scope, and a viable Lean representation. They originate from established quantum results, library development objectives, or source statements whose assumptions can be made explicit. Duplicates, purely auxiliary declarations, and claims that cannot be expressed within the fixed task format are excluded. The admitted tasks are then assigned to one of six fields. QAlg-Bench uses state and operator methods (SOM), circuit and algebraic algorithms (CAA), and

simulation, signal processing, and learning (SSL). QIT-Bench uses quantum channels and representations (QCR), operator and state geometry, symmetry, and distinguishability (GSD), and quantum information measures and entanglement (IME). Membership in a field is fixed by the task's scope, and each field's difficulty denominator is the sum of its task difficulties. The complete inventories appear in Appendix A.

A Lean task contains a theorem statement and the supporting definitions needed to state it. Researchers determine the mathematical scope, assumptions, and conclusion, and agents assist with the Lean translation. The definitions introduce benchmark-local objects that are not supplied by imported libraries, and the statement contains the theorem signature whose body the agent must complete. The evaluated tasks provide no hints.

Every admitted task passes the same automated checks. Its statement and supporting definitions are assembled in the fixed Lean environment and compiled by Lean before inclusion in the benchmark. This checking stage applies to every task and denotes automated compilation and screening; it does not imply that compilation guarantees mathematical fidelity or that every statement receives the targeted manual comparison described below. Submission-side acceptance applies the same discipline: the independent final verification recompiles the submitted theorem in the fixed environment and accepts it only when the proof introduces no new axioms, contains no `sorry` placeholder, and leaves all files outside the theorem body unchanged.

The same benchmark task is used for both evaluation conditions. The baseline condition supplies the statement and supporting definitions. LAD supplies the same task together with a verified collection of library references available only for consultation. Each result records a fixed suite version, model setting, access condition, and execution setting, which support descriptive aggregation at the run level.

These construction steps establish the common mechanical checks. The next subsection addresses the separate question of whether selected formal statements faithfully represent their motivating mathematics.

3.2 Quality assurance and semantic validation

Lean's kernel guarantees that an accepted proof has the stated theorem as its type, relative to the imported environment [28, 29]. It does not, however, guarantee that the statement faithfully captures the scientific claim that motivated the task. A benchmark can therefore be mechanically valid while remaining semantically misleading: an omitted hypothesis, an incorrect type or function encoding, or a weakened conclusion can cause the formal target to diverge from the intended mathematics [26]. We therefore treat Lean compilation and the automated checks applied to every task as universal mechanical guarantees, while regarding semantic fidelity as a separate validity question.

For selected statements whose translation carries elevated semantic risk, we perform a manual comparison between the formal signature, its supporting definitions, and the available mathematical specification. This comparison examines the relevant objects and quantifiers, explicit and implicit assumptions, dimensional constraints, index and tensor ordering, and the direction and strength of the conclusion. Any discrepancies identified in this process are resolved before the task is included in the benchmark.

This targeted comparison supplements the automated checks applied to every task. Because it is performed only on selected high-risk statements, it is not intended to define a benchmark-wide rate of semantic validity. With the benchmark construction, automated checks, and targeted semantic safeguards in place, the next section evaluates the recorded model runs.

4 Evaluation

The evaluation addresses three complementary questions: how objective proof completion varies across models and access conditions, what resource cost accompanies the observed scores, and whether suite-level aggregates conceal field-specific strengths or weaknesses.

We evaluate one recorded run for each suite–model–condition combination. Every run records its suite version and problem set, model and agent setting, access condition, budget, retry policy, and execution environment. These settings are fixed within a run, although the historical baseline and LAD pairs are not fully matched across all execution details. QAlg-Bench contributes 36 tasks and QIT-Bench contributes 40, giving 16 combinations. The runs also differ in agent tool: GPT-5.5 uses a different agent tool from the other three models (Table 6), so each combination measures a model and agent tool jointly, and cross-model orderings compare these joint settings. The exact model and agent settings are listed in Appendix A.

For task i, let $d_i \in \{1, \dots, 10\}$ be the difficulty fixed before model execution, and let $v_i \in \{0, 1\}$ be the recorded acceptance indicator for the final submitted theorem under the common Lean acceptance checks. We define the score as

$$\text{score} = 100 \frac{\sum_i d_i v_i}{\sum_i d_i}. \tag{1}$$

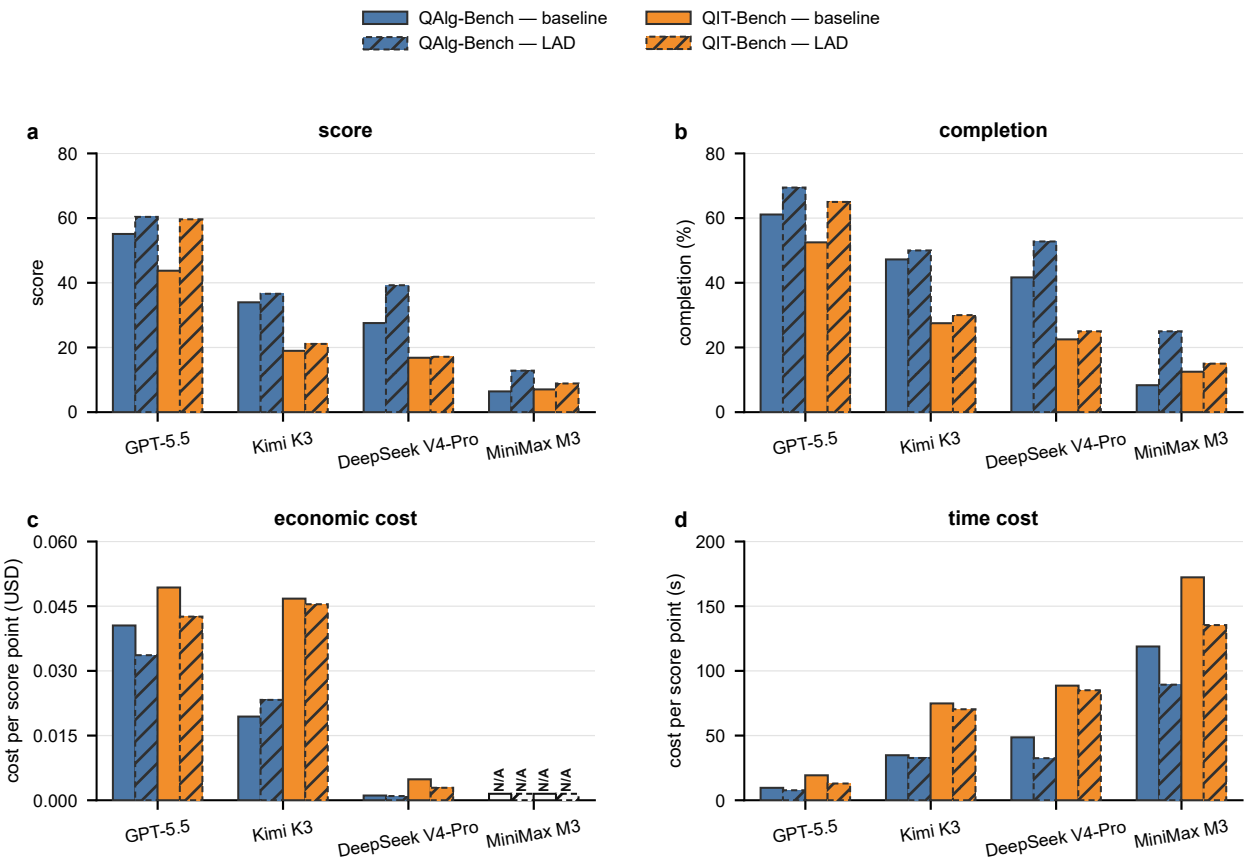

Figure 2: Verified performance and per-score costs across QAlg-Bench and QIT-Bench. Panels (a,b) show the difficulty-weighted score and completion, respectively. Panels (c,d) show economic cost in USD per score point and time cost in seconds per score point, respectively; lower values indicate lower cost. Blue and orange identify QAlg-Bench and QIT-Bench. Solid, unhatched bars indicate the baseline condition, while dashed, diagonally hatched bars indicate LAD. Black N/A labels mark unavailable economic-cost values.

Completion is the unweighted fraction of tasks with $v_i = 1$. The score therefore weights verified tasks by their pre-exam difficulty, whereas completion treats every task equally. A model-call timeout does not itself set $v_i = 0$: after the invocation ends, an independent Lean verification checks the final submission. Four tasks in the snapshot reached the timeout and nevertheless passed this final check. Neither metric assesses intermediate proof progress.

Economic cost is the mean API list-price-equivalent USD among tasks with reconstructable prices, divided by the difficulty-weighted score for those tasks. Time cost is the mean model invocation time per task over the complete suite divided by the overall score. Model invocation time is the elapsed duration of the complete agent invocation, including Lean checks initiated by the agent; it excludes the independent final verification performed after the invocation ends. The general pricing schedule was fixed on 2026-07-20, while the Kimi K3 prices were fixed on 2026-07-22. The units are USD and seconds per score point, and lower values indicate lower cost. Both ratios require a positive score denominator; unavailable ratios are reported as N/A, with zero reserved for observed zeros.

4.1 Overall performance

Figure 2(a,b) compares the 16 recorded results. GPT-5.5 has the highest observed score in every suite–condition combination. In all eight historical within-model comparisons, the LAD result has higher score and completion than the baseline result. The magnitude varies substantially: DeepSeek V4-Pro, for example, shows a much larger separation between conditions on QAlg-Bench than on QIT-Bench. These single-run differences describe the recorded runs; the unmatched execution details prevent their interpretation as estimates of an access-condition effect.

Figure 2(c,d) shows that the cost ordering differs from the performance ordering. DeepSeek V4-Pro has

Table 1: Relative change from baseline to LAD by suite and model. Every metric column uses $(L - B)/B$ computed from unrounded values. Positive score and completion values indicate higher verified performance; positive economic- and time-cost values indicate higher cost per score point, and negative values indicate lower cost per score point. Economic cost divides mean API list-price-equivalent USD among tasks with reconstructable prices by the corresponding score. Time cost divides mean model invocation seconds per task over the full suite by the overall score. A black em dash denotes an unavailable observed metric. All eight observed score changes are positive, but their magnitudes and the cost changes vary by model and suite.

Suite	Model	Score	Completion	Economic cost	Time cost
QAlg-Bench	GPT-5.5	+9.6%	+13.6%	-17.0%	-19.5%
	Kimi K3	+7.8%	+5.9%	+19.9%	-6.1%
	DeepSeek V4-Pro	+42.5%	+26.7%	-11.1%	-33.1%
	MiniMax M3	+100.0%	+200.0%	—	-24.8%
QIT-Bench	GPT-5.5	+36.4%	+23.8%	-13.7%	-32.9%
	Kimi K3	+11.3%	+9.1%	-2.8%	-6.0%
	DeepSeek V4-Pro	+1.8%	+11.1%	-40.4%	-4.0%
	MiniMax M3	+26.1%	+20.0%	—	-21.4%

the lowest reported economic cost, while GPT-5.5 has the lowest observed time cost. These quantities are costs per score point. Total expenditure, total elapsed time, and intrinsic model speed are separate quantities.

Table 1 expresses each available LAD–baseline difference relative to its baseline value as $(L - B)/B$, using unrounded condition-level values. Positive score or completion entries denote a higher LAD value. For economic and time cost, positive entries denote higher cost per score point and negative entries denote lower cost per score point. The economic-cost entries compare the corresponding condition-level ratios. A black em dash marks an unavailable observed comparison.

Relative changes should be read together with their absolute baseline values. MiniMax M3 doubles its QAlg-Bench score from 6.4 to 12.8 from a low baseline, while the larger absolute increases for GPT-5.5 on QIT-Bench and DeepSeek V4-Pro on QAlg-Bench are 15.9 and 11.7 score points. These ratios describe the recorded historical runs; matched reruns are required to isolate the access-condition difference. The absolute values are reported in Appendix B.

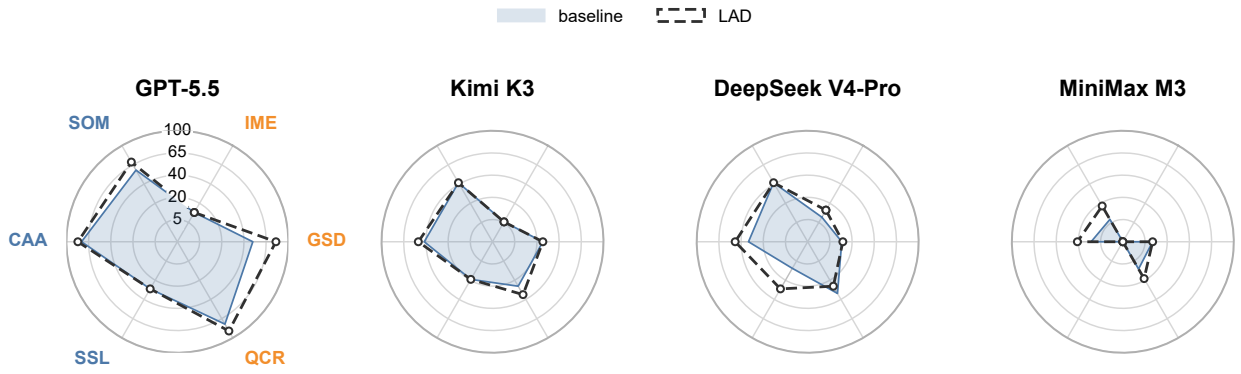

Figure 3: Difficulty-normalized field profiles for GPT-5.5, Kimi K3, DeepSeek V4-Pro, and MiniMax M3, from left to right. The six axes represent the QAlg-Bench and QIT-Bench fields defined in Tables 2 and 3; exact values are reported in Table 8. All panels use the same radial mapping, with equally spaced rings labelled 5, 20, 40, 65, and 100. Baseline is shown as a translucent filled profile with a solid outline, while LAD is shown as an unfilled dashed profile with hollow nodes. Each axis aggregates 10 to 17 tasks, so a single verified task can visibly move a field score.

4.2 Field-level capability profiles

We partition each suite into three fields and apply Equation (1) within each partition. QAlg-Bench comprises state and operator methods (SOM), circuit and algebraic algorithms (CAA), and simulation, signal processing,

and learning (SSL), with fixed difficulty denominators 67, 105, and 93. QIT-Bench comprises quantum channels and representations (QCR), operator and state geometry, symmetry, and distinguishability (GSD), and quantum information measures and entanglement (IME), with denominators 80, 138, and 109. These six disjoint partitions cover the 76 tasks exactly. Their unequal denominators mean that the same number of additional verified tasks can produce different field-score changes, depending on task difficulty and field size. Each field contains between 10 and 17 tasks, and a single verified task of difficulty 10 can move a field score by up to roughly 15 points at the smallest denominator, so the field profiles should be read together with these counts.

Figure 3 shows that suite-wide scores mask persistent field differences. GPT-5.5 has the strongest observed profile overall, but its QIT-Bench scores are much higher on QCR and GSD than on IME. Across models, SSL on QAlg-Bench and IME on QIT-Bench remain comparatively weak, so observed verification success is unevenly distributed across the fields in both suites. The common radial mapping permits axis-by-axis comparisons across panels, but polygon area is not used in any aggregate or ranking.

Historical baseline–LAD differences are also localized. GPT-5.5's largest field change is a 30.4-score-point rise in GSD on QIT-Bench. Kimi K3's CAA net change is +6.7 score points, comprising two tasks verified only under LAD and one task verified only under baseline. Its QCR net change on QIT-Bench is +8.8 points from one additional verified task and no regression; its other fields are unchanged. DeepSeek V4-Pro rises by 13.3 and 18.3 points in CAA and SSL on QAlg-Bench but declines by 7.5 points in QCR on QIT-Bench. MiniMax M3 remains at 0.0 in SSL on QAlg-Bench and IME on QIT-Bench under both conditions.

The field profiles explain why a positive suite-wide change need not be uniform. DeepSeek V4-Pro's local QCR decline on QIT-Bench, for example, coexists with a suite-wide QIT-Bench score change from 16.8 to 17.1 because changes in other fields offset it. Figure 3 should therefore be read as a decomposition of Figure 2 and Table 1, with exact local values reported in Appendix B. The following two sections describe the mathematical content behind these field profiles and present representative tasks from each suite.

5 Lean-QuantumAlg-Bench

To benchmark the proof capabilities of AI systems in quantum algorithms, we design QAlg-Bench, which contains 36 theorem-proving tasks drawn from the mathematics of quantum algorithms and uses the LeanQuantumAlg library [9, 10], developed by an overlapping subset of the present authors. The tasks are organized into three complementary fields, covering state and operator methods, circuit identities, number-theoretic algorithms, block encodings, and quantum signal processing. Each benchmark task specifies a Lean 4 theorem signature together with the supporting definitions available to the prover; success is determined by whether the submitted theorem body compiles in the fixed Lean environment.

5.1 Task fields

The benchmark groups eight mathematical scopes into three fields in Lean-QuantumAlg-Bench. Table 2 summarizes their meanings and problem counts.

The three fields emphasize complementary capabilities. State and operator methods exercise matrix algebra, spectral arguments, and reasoning about measurements. Circuit and algebraic algorithms combine gate composition, Fourier methods, order finding, and hidden-subgroup arguments. Simulation, signal processing, and learning covers product-formula bounds, block encodings, quantum signal processing, and quantum singular value transformation [48,49], together with parameterized-circuit trainability [50]. These fields exercise different proof skills, including compositional tensor-product reasoning, number-theoretic preconditions, and ancilla-register management. The full task inventory appears in Appendix A.

5.2 Representative problems

We present three tasks chosen to illustrate distinct mathematical interfaces. For each, we give the source-level statement, the Lean 4 formalization, and the main issues a prover must navigate.

Problem 1: Walsh–Hadamard decoding of a Boolean character (circuit and algebraic algorithms)

Source statement. The Bernstein–Vazirani algorithm [46] uses this Walsh–Hadamard decoding identity. For a fixed bitstring $s \in \{0,1\}^n$, define the Boolean character state

$$|\chi_s\rangle = 2^{-n/2} \sum_{y \in \{0,1\}^n} (-1)^{s \cdot y} |y\rangle,$$

Table 2: Task fields and scopes in QAlg-Bench.

Field	Meaning	Scope	Count
SOM	state and operator methods	state, operator, measurement, and matrix algebra	10
CAA	circuit and algebraic algorithms	oracle and elementary quantum-circuit identities	9
		quantum Fourier transform, phase estimation, and order finding	4
		hidden subgroup and representation theory	3
SSL	simulation, signal processing, and learning	product formulas and norm bounds	3
		linear combinations of unitaries (LCU), block encoding, and quantum singular value transformation primitives	2
		quantum signal processing, parameter-shift rules, and trace estimation	4
		parameterized circuits, Lie algebra, and trainability	1
		Total	**36**

where $s \cdot y = \sum_j s_j y_j \bmod 2$. Prove that

$$H^{\otimes n}|\chi_s\rangle = |s\rangle.$$

Formalization. The Lean statement (Listing 1) is concise: one bitstring argument and one equation. Its mathematical structure is character orthogonality over $\mathbb{F}_2^n$.

Listing 1: Walsh–Hadamard decoding of a Boolean character

```
namespace QAlgBench.WalshHadamardBooleanCharacter
noncomputable section
open QAlgBench Gate
variable {n : ℕ}

/-- H^{⊗n} |χ_s⟩ = |s⟩. -/
theorem main (s : Fin n → Bool) :
    HilbertOperator.applyVec hadamardTensorOp (chiState s)
      = (PureState.ket (R := BitReg n) s
          : StateVector (BitReg n)) := by
  sorry
```

The statement is short, but it already exercises library-specific skills. The prover must use the domain types `BitReg n` (an n-qubit register), `StateVector`, and `PureState.ket`, and handle the coercion from the pure-state constructor to the ambient state-vector type. Operator application goes through `HilbertOperator.applyVec`, so the prover must operate at the correct API layer above bare matrix multiplication.

Problem 2: Eigenvectors of modular multiplication in Shor's algorithm (circuit and algebraic algorithms)

Source statement. Modular multiplication supplies the periodic unitary underlying Shor's order-finding algorithm [1]. Kitaev's eigenvalue-measurement formulation [51] supplies the corresponding phase-estimation viewpoint. Let $x, N \in \mathbb{N}^+$ with $x < N$ and $\gcd(x, N) = 1$. Let r be the multiplicative order of x modulo N. Define $U_x|y\rangle = |xy \bmod N\rangle$ and, for $s \in \{0, \ldots, r-1\}$,

$$|\psi_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} |x^k \bmod N\rangle.$$

Prove that $|\psi_s\rangle$ is an eigenvector of U_x with eigenvalue $e^{2\pi i s/r}$.

Formalization. In Lean (Listing 2) the two-line textbook statement expands to a theorem with six explicit hypotheses and three data arguments.

Listing 2: Eigenvectors of modular multiplication in Shor's algorithm

```
namespace QAlgBench.EigenvectorsModularMultiplicationShor
noncomputable section
open QAlgBench
variable {N : ℕ} [NeZero N]

theorem main (x : ℕ)
    (hx_pos : 0 < x) (hx_lt : x < N)
    (hcoprime : Nat.Coprime x N)
    (hunit : IsUnit (x : ZMod N))
    (r : ℕ) (hr : r = orderOf hunit.unit)
    (s : ℕ) (hs : s < r) :
    HilbertOperator.applyVec (UxOp (N := N) x)
      (psiS (N := N) x r s) =
      Complex.exp (2 * Real.pi * Complex.I
        * (s : ℂ) / (r : ℂ))
        • psiS (N := N) x r s := by
  sorry
```

The problem combines number-theoretic and quantum-mechanical reasoning. The prover must use Mathlib's `Nat.Coprime`, `ZMod N`, `IsUnit`, and `orderOf` alongside the quantum types `HilbertOperator` and `psiS`, and negotiate the coercion $\mathbb{N} \to$ `ZMod N` $\to$ `Units`. The hypothesis `hunit` (that x is a unit in $\mathbb{Z}/N\mathbb{Z}$) is derivable from coprimality but appears explicitly because `orderOf` requires a named unit. The remaining hypotheses (positivity, the bound $x < N$, coprimality, the order definition, and the index bound) are implicit in the textbook phrase "let r be the order of x mod N."

Problem 3: LCU block encoding from PREPARE and SELECT (simulation, signal processing, and learning)

Source statement. This problem isolates an algebraic step in the method of linear combinations of unitaries [47]. The circuit composed of PREPARE, SELECT, and UNPREPARE appears in Hamiltonian simulation by a truncated Taylor series [52]. The resulting unitary is a block encoding whose projected block equals A/α [48]. Let $\{\alpha_\ell > 0\}$ be positive coefficients and $\{U_\ell\}$ unitaries on a system Hilbert space, and assume that the label states exhaust the ancilla label basis. Define $A = \sum_\ell \alpha_\ell U_\ell$, $\alpha = \sum_\ell \alpha_\ell$, and $\mathrm{SELECT} = \sum_\ell |\ell\rangle\langle\ell| \otimes U_\ell$. Let a PREPARE unitary P satisfy $P|0\rangle = \alpha^{-1/2} \sum_\ell \sqrt{\alpha_\ell}|\ell\rangle$. Set $W = (P^\dagger \otimes I)\,\mathrm{SELECT}\,(P \otimes I)$. Prove that $(\langle 0| \otimes I)\, W\, (|0\rangle \otimes I) = A/\alpha$.

Formalization. The Lean statement (Listing 3) manages an ancilla–system tensor product, an embedding of the LCU label set into the ancilla basis, and the projected-block extraction that defines the block encoding.

Listing 3: LCU block encoding from PREPARE and SELECT

```
namespace QAlgBench.LCUBlockEncodingPrepareSelect
noncomputable section
open QAlgBench
open scoped BigOperators
variable {a n : ℕ} {L : Type*} [Fintype L] [DecidableEq L]

theorem ProjectedBlock.main
    (embed : L ↪ Fin (2 ^ a))
    (αcoeff : L → ℝ) (U : L → HilbertOperator (Qubits n))
    (P : Gate (Qubits a)) (W : Gate (Qubits (a + n)))
    (hαcoeff_pos : ∀ ℓ, 0 < αcoeff ℓ)
    (hU_unitary : ∀ ℓ, U ℓ ∈ Matrix.unitaryGroup (Fin (2^n)) ℂ)
    (hPrepare : ∀ j : Fin (2 ^ a),
      (P : HilbertOperator (Qubits a)) j 0 =
        ((Real.sqrt (coeffSum αcoeff))⁻¹ : ℝ) *
          ∑ ℓ, if embed ℓ = j
               then (Real.sqrt (αcoeff ℓ) : ℝ) else 0)
    (hW_eq : (W : HilbertOperator (Qubits (a + n))) =
```

```
    HilbertOperator.tensor
      (P : HilbertOperator (Qubits a)).conjTranspose
      (1 : HilbertOperator (Qubits n))
    * selectOp embed U
    * HilbertOperator.tensor
        (P : HilbertOperator (Qubits a))
        (1 : HilbertOperator (Qubits n))) :
  projectedBlock a n (W : HilbertOperator (Qubits (a + n))) =
    ((coeffSum αcoeff : ℂ)⁻¹) • weightedSum αcoeff U := by
sorry
```

The benchmark-local `selectOp` vanishes outside the image of the label embedding. The theorem also assumes that W is a unitary gate equal to the PREPARE–SELECT composition. These hypotheses are therefore jointly realizable only when the embedded labels exhaust the ancilla basis. The formal statement proves the projected-block identity under this boundary; it does not construct the usual unitary identity extension on unused labels.

The type `Qubits (a + n)` encodes a composite Hilbert space of dimension 2^{a+n}, and its tensor-product structure is accessed through `HilbertOperator.tensor`, abstracting away the explicit Kronecker product. The PREPARE hypothesis is stated as a column-zero condition on the unitary matrix, so the prover must connect the gate-level and matrix-level descriptions. The label embedding $L \hookrightarrow \mathrm{Fin}(2^a)$ is formalized with Lean's `Function.Embedding`, forcing the prover to reason about injective maps between finite types. The task consequently combines quantum-algorithm content, linear algebra, and nontrivial use of Lean's type system.

These three examples illustrate why QAlg-Bench does not reduce to one proof idiom. Walsh–Hadamard decoding emphasizes state and operator interfaces, modular multiplication combines quantum structure with number-theoretic coercions and hypotheses, and LCU block encoding adds tensor composition and finite-type embeddings. Their different interfaces provide mathematical context for the field-level results in Section 4, where suite-wide results are decomposed to show which mathematical and library interfaces remain difficult.

6 Lean-QIT-Bench

To benchmark the proof capabilities of AI systems in quantum information theory, we design QIT-Bench, which contains 40 theorem-proving tasks drawn from quantum information theory and uses the Lean-QIT library [8], developed by an overlapping subset of the present authors. Its three fields cover channel representations; operator and state geometry, symmetry, and distinguishability; and information measures and entanglement. Each benchmark task specifies a Lean 4 theorem signature together with the supporting definitions available to the prover; success is determined by whether the submitted theorem body compiles in the fixed Lean environment.

6.1 Task fields

The benchmark groups five mathematical scopes into three fields in Lean-QIT-Bench. Table 3 summarizes their meanings and problem counts.

The three fields emphasize complementary forms of reasoning. Quantum channels and representations asks the prover to move among Kraus, Choi, Stinespring, and axiomatic CPTP descriptions. Operator and state geometry, symmetry, and distinguishability combines symmetry computations with trace norm, fidelity, continuity bounds, and gentle measurement. Quantum information measures and entanglement covers entropy, mutual information, and finite-resource information measures [3, 38]; the Holevo and Fano bounds [3]; and entanglement concentration [53]. The corresponding proof skills include finite-index bookkeeping, combinatorial dimension formulas, chains of operator inequalities, spectral majorization, and asymptotic ε/δ reasoning over local operations and classical communication (LOCC) protocols. The full task inventory appears in Appendix A.

6.2 Representative problems

We present three tasks chosen to exercise distinct prover capabilities: type-level index bookkeeping, real and operator inequality chaining, and asymptotic resource-theoretic reasoning. For each, we give the source-level statement, the Lean 4 formalization, and the main issues a prover must navigate.

Table 3: Task fields and scopes in QIT-Bench.

Field	Meaning	Scope	Count
QCR	quantum channels and representations	channel, Choi, Kraus, and Stinespring-style representations	11
GSD	operator and state geometry, symmetry, and distinguishability	mixed-unitary channels, Werner–Holevo maps, and symmetry projectors	7
		trace norm, fidelity, continuity bounds, and gentle measurement	10
IME	quantum information measures and entanglement	entropy, source coding, Holevo/Fano bounds, and convexity	9
		entanglement conversion and hypothesis-testing tools	3
		Total	**40**

Problem 1: Partial trace as a completely positive trace-preserving map (quantum channels and representations)

The partial trace connects composite-system states to reduced states and channel representations. This task isolates the translation of an abstract linear map into an explicit Kraus decomposition [3]. Its statement is quantified over two arbitrary finite types `a` and `b`, so a valid proof must hold uniformly across finite bipartite dimensions rather than for one fixed matrix size.

Source statement. Let $\mathcal{H}_A$ and $\mathcal{H}_B$ be finite-dimensional Hilbert spaces. The partial trace over B is the linear map

$$\mathrm{Tr}_B : \mathcal{L}(\mathcal{H}_A \otimes \mathcal{H}_B) \to \mathcal{L}(\mathcal{H}_A).$$

Prove that Tr_B is completely positive and trace-preserving by giving an explicit Kraus representation after fixing an orthonormal basis of $\mathcal{H}_B$.

Formalization. The Lean statement (Listing 4) packages the partial trace as a `MatrixMap`, defines Kraus operators indexed by the environment type `b`, and asserts complete positivity through the Kraus form together with trace preservation through the library predicate.

Listing 4: Partial trace as a completely positive trace-preserving map

```
namespace QITBench.PartialTraceCompletelyPositiveTracePreservingMap
noncomputable section
open QITBench

variable {a b : Type*} [Fintype a] [DecidableEq a] [Fintype b] [DecidableEq b]

noncomputable def partialTraceBMap : MatrixMap (a × b) a where
  toFun := partialTraceB (a := a) (b := b)
  map_add' X Y := partialTraceB_add X Y
  map_smul' c X := partialTraceB_smul c X

noncomputable def partialTraceBKraus (k : b) : Matrix a (a × b) ℂ :=
  fun i p => if p.1 = i ∧ p.2 = k then 1 else 0

/-- The partial trace over B is a CPTP map with explicit Kraus operators. -/
theorem main :
    partialTraceBMap (a := a) (b := b) =
        MatrixMap.ofKraus (partialTraceBKraus (a := a) (b := b)) ∧
      MatrixMap.IsTracePreserving (partialTraceBMap (a := a) (b := b)) := by
  sorry

end
end QITBench.PartialTraceCompletelyPositiveTracePreservingMap
```

The mathematics is elementary, and the real demand is organizational. The prover must reconcile three views of one object: the partial trace as a concrete sum over an environment index, as a linear map between matrix spaces, and as a Kraus decomposition. Complete positivity is discharged through `MatrixMap.ofKraus` applied to the explicit operators `partialTraceBKraus`, and trace preservation through the separate predicate `MatrixMap.IsTracePreserving`; the two views are then reconciled by pointwise agreement on matrix entries. The indexing type `b` stands in for the chosen orthonormal basis of $\mathcal{H}_B$, so the proof is parameterized by an arbitrary finite type. The problem therefore tests index bookkeeping over product types and the discipline of staying at the library's `MatrixMap` abstraction above bare matrix multiplication.

Problem 2: Fuchs–van de Graaf inequalities (operator and state geometry, symmetry, and distinguishability)

The Fuchs–van de Graaf inequalities [54] relate trace distance and root fidelity and are standard tools in continuity and distinguishability arguments. This task probes a capability distinct from Problem 1: chaining real and operator inequalities through functional calculus on complex matrices while working directly with square roots and traces. The statement is a conjunction, and its two directions impose different proof obligations on the trace norm.

Source statement. For density operators $\rho, \sigma \in \mathcal{S}(\mathcal{H})$, define the trace distance and use the root-fidelity convention

$$D(\rho,\sigma) = \frac{1}{2}\|\rho-\sigma\|_1, \qquad F(\rho,\sigma) = \mathrm{Tr}\sqrt{\sqrt{\rho}\,\sigma\sqrt{\rho}}.$$

Prove the two-sided bounds:

$$1 - F(\rho,\sigma) \le D(\rho,\sigma) \le \sqrt{1-F(\rho,\sigma)^2}.$$

Formalization. The Lean statement (Listing 5) defines the trace norm, trace distance, and root fidelity explicitly from matrix square roots and traces, then states the two-sided inequality as a conjunction.

Listing 5: Fuchs–van de Graaf inequalities

```
namespace QITBench.FuchsVanDeGraafInequalities
noncomputable section
open QITBench
open scoped ComplexOrder MatrixOrder

variable {n : ℕ}

noncomputable def matrixSqrt (A : CMatrix (Fin n)) : CMatrix (Fin n) :=
  CFC.sqrt A

noncomputable def traceNorm (A : CMatrix (Fin n)) : ℝ :=
  Complex.re (Matrix.trace (matrixSqrt (A.conjTranspose * A)))

noncomputable def traceDistance (rho sigma : CMatrix (Fin n)) : ℝ :=
  (1 / 2 : ℝ) * traceNorm (rho - sigma)

noncomputable def unsquaredFidelity (rho sigma : CMatrix (Fin n)) : ℝ :=
  Complex.re (Matrix.trace (matrixSqrt (matrixSqrt rho * sigma * matrixSqrt rho)))

/-- 1 - F(rho,sigma) <= D(rho,sigma) <= sqrt(1 - F(rho,sigma)^2). -/
theorem main (rho sigma : State (Fin n)) :
    let D := traceDistance rho.matrix sigma.matrix
    let F := unsquaredFidelity rho.matrix sigma.matrix
    1 - F ≤ D ∧ D ≤ Real.sqrt (1 - F ^ 2) := by
  sorry

end
end QITBench.FuchsVanDeGraafInequalities
```

The inequalities are mathematically standard, and the formal difficulty is concentrated in the functional calculus. The prover must build the trace norm, trace distance, and root fidelity directly from `CFC.sqrt` and `Matrix.trace`, taking real parts of complex traces throughout. Both bounds demand careful handling

of positivity under the square root and of the singular values of $\rho - \sigma$. Standard proofs derive the lower bound from a measurement-based classical variational characterization and obtain the upper bound from purification, Uhlmann's theorem, and monotonicity of trace distance [3]. The problem therefore tests sustained manipulation of real and operator inequalities inside an explicit formalization of the matrix square root.

Problem 3: Achievability of entanglement concentration (quantum information measures and entanglement)

Entanglement concentration establishes an achievable LOCC rate for converting many copies of a bipartite pure state into maximally entangled pairs [53]. The task combines tensor powers, a product-Kraus separable-operation abstraction motivated by the source-level LOCC requirement, maximally entangled states of a prescribed rank, and convergence of a fidelity sequence, all under a rate strictly below the Schmidt entropy. It therefore tests reasoning about an infinite family of protocols together with Mathlib's topology interface.

Source statement. Let

$$|\psi\rangle_{AB} = \sum_{x \in \mathcal{X}} \sqrt{p(x)} |x\rangle_A |x\rangle_B$$

be a bipartite pure state with Schmidt coefficients $\{p(x)\}_{x \in \mathcal{X}}$. Its entanglement entropy is

$$E(\psi) = S(\psi_A) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x).$$

Let $|\Phi_M\rangle_{AB} = M^{-1/2} \sum_{j=1}^{M} |j\rangle_A |j\rangle_B$ be the standard maximally entangled state of Schmidt rank M. Prove that for every rate $0 \le R < E(\psi)$ there exists a sequence of LOCC channels $\{\Lambda_n\}_{n \ge 1}$ and positive integers $M_n = \lfloor 2^{nR} \rfloor$ such that

$$\lim_{n \to \infty} F\left(\Lambda_n\left(|\psi_{AB}\rangle\langle\psi_{AB}|^{\otimes n}\right), |\Phi_{M_n}\rangle\langle\Phi_{M_n}|\right) = 1,$$

where $F(\rho, \sigma) = \|\sqrt{\rho}\sqrt{\sigma}\|_1$ denotes the quantum fidelity.

Formalization. The Lean statement (Listing 6) quantifies over a probability distribution, a bipartite pure state with prescribed Schmidt coefficients, a rate below the Schmidt entropy, a target-rank function, and a family represented by `SEPProtocol`. This type stores a product-Kraus separable operation. Every finite-round LOCC channel induces such a representation, but a general separable operation need not admit a finite-round LOCC implementation [3, 55]. Consequently, the formal target is a relaxation of the source achievability claim: it verifies convergence within the broader separable-operation class. The benchmark definition `schmidtEntropy` uses $\log_2$, and `targetRankAtRate R n` is $\lfloor 2^{nR} \rfloor$, so the entropy, rate, and target rank use the same bit convention. The library function `quantumFidelity` implements the same root-fidelity convention used in Problem 2, rather than its square.

Listing 6: Achievability of entanglement concentration

```
namespace QITBench.AchievabilityEntanglementConcentration
noncomputable section
open QITBench
open scoped BigOperators
open QITBench.OneShot

variable {X : Type*} [Fintype X] [DecidableEq X]

/-- For every rate R below the entanglement entropy, there is a sequence of
product-Kraus separable protocols whose output converges in fidelity to a
maximally entangled state of rank M_n = floor(2^{nR}). -/
theorem main
    (p : X → ℝ)
    (psi : PureVector (X × X))
    (hp : IsProbabilityDistribution p)
    (hpsi : HasSchmidtCoefficients psi p) :
    ∀ R : ℝ,
      0 ≤ R →
        R < schmidtEntropy p →
          ∃ M : ℕ → ℕ,
            ∃ protocols :
```

```
              (n : ℕ) →
                SEPProtocol
                  (TensorPower X n) (TensorPower X n)
                  (Fin (M n)) (Fin (M n)),
              (∀ n, 0 < M n) ∧
                (∀ n, M n = targetRankAtRate R n) ∧
                  Filter.Tendsto
                    (fun n =>
                      quantumFidelity
                        ((protocols n).channel.applyState
                          (psi.state.tensorPowerBipartite n)).matrix
                        (maximallyEntangledDensity (M n)))
                    Filter.atTop
                    (nhds 1) := by
  sorry

end
end QITBench.AchievabilityEntanglementConcentration
```

The prover must assemble several abstractions from Lean-QIT: tensor powers of a bipartite state through `psi.state.tensorPowerBipartite`, the product-Kraus/ separable-operation type `SEPProtocol`, the maximally entangled density matrix `maximallyEntangledDensity`, and the `quantumFidelity` between each protocol output and its target. The type `SEPProtocol (TensorPower X n) (TensorPower X n) (Fin (M n)) (Fin (M n))` fixes the input and output spaces of the n-th protocol, and `targetRankAtRate R n` supplies $M_n = \lfloor 2^{nR} \rfloor$ explicitly. The conclusion is phrased with `Filter.Tendsto`, so the proof combines quantum-information reasoning with Mathlib's topology library. The statement itself is an existential over an infinite family of protocols, and the proof must reason about every family member uniformly. The task illustrates the suite's inclusion of resource-theoretic asymptotic statements alongside finite-dimensional identities.

These examples likewise span distinct Lean-QIT interfaces, from finite-dimensional channel representations to operator inequalities and asymptotic resource-theoretic statements. Their obligations combine indexed finite types, matrix and operator norms, tensor powers, topology, and quantified protocol families. This diversity provides mathematical context for the field-level scores reported alongside the suite-wide results in Section 4.

7 Discussion and Conclusion

7.1 Discussions on the limitations

The reported results have four main validity limits that warrant further study: corpus coverage and difficulty calibration, semantic fidelity of the formal statements, comparability of the recorded runs, and availability of economic-cost records. Together, these limits constrain how far the observed scores and costs can be generalized beyond the present evaluation. These limitations make the reported results an initial measurement of the recorded benchmark runs, rather than a population estimate or a causal comparison of access conditions.

At the corpus level, the two suites sample theorem-sized tasks from selected parts of quantum algorithms and quantum information under the current Lean library interfaces. Their 36 and 40 tasks do not provide complete coverage of either domain, and the six author-defined fields aggregate areas of unequal size and maturity. Task selection, theorem granularity, and the availability of reusable definitions all affect what can enter the benchmark. Difficulty is assigned before model execution through two rubric-based reviews covering proof structure, theory dependencies, Lean engineering, and mathematical technique; scores differing by more than one point are adjudicated. This procedure prevents result-dependent reweighting, but the 1–10 assignments are not objective measurements of proof complexity. Some benchmark statements may overlap with material in model training corpora, and the present evaluation cannot determine such overlap. miniCTX [25] uses recency-based theorem selection to mitigate contamination; this benchmark does not.

Beyond corpus coverage, mechanical validity does not eliminate semantic risk. Refs. [26, 27] show that kernel acceptance alone does not establish fidelity to the intended informal problem or robustness of the evaluation harness. In this benchmark, for example, the entanglement-concentration task represents protocols by product-Kraus separable operations rather than a full finite-round LOCC syntax, so its formal target is a relaxation of the motivating achievability claim. Lean compilation and automated checks apply to every task, while the separate semantic comparison described in Section 3.2 covers only selected statements and does not

support a benchmark-wide semantic-validity rate. Residual translation risk therefore remains.

For the fixed task set, the evaluation is bounded by the tested models, agent tools, budgets, hardware, suite versions, and access policies. Each suite–model–condition combination contains one run, so the results provide no estimate of stochastic variation. Historical baseline and LAD executions are not fully matched, which limits paired interpretation even under a common scoring rule. LAD adds a fixed reference library available only for consultation, but its semantic overlap with the benchmark targets is unmeasured. These results therefore do not characterize all theorem provers, retrieval systems, or forms of tool access.

Economic-cost accounting is also incomplete for some combinations because historical runs did not consistently retain the usage and pricing information needed to reconstruct API list-price-equivalent cost. Score, completion, and model invocation time remain available for every combination, but missing economic values limit cost comparisons. Provider-specific pricing assumptions and historical execution differences impose further constraints.

7.2 Conclusion

In this work, we introduced QAlg-Bench and QIT-Bench, two formal theorem-proving benchmarks for quantum algorithms and quantum information, comprising 76 tasks across six fields. A shared task contract, difficulty weights fixed before model execution, and Lean-based acceptance checks together define a rigorous and fully reproducible evaluation protocol that can be applied across different mathematical domains and library interfaces. Across four models and two access conditions, the highest observed scores are 60.4 on QAlg-Bench and 59.6 on QIT-Bench, showing that current models still struggle with certain structured quantum-theorem tasks. Verified proofs are unevenly distributed across fields, and the accompanying monetary and wall-clock costs provide a complementary view of model performance beyond pass rates alone. These single-run baselines establish an initial reference point for research on agentic provers in quantum information science and identify library access as a nontrivial variable whose effects warrant further study.

Our findings motivate matched repeated runs, broader task coverage, and more systematic semantic review. More importantly, the field-level weaknesses uncovered here make a strong case for proof agents that combine quantum-domain reasoning with more reliable use of library definitions and references. Taken together, QAlg-Bench and QIT-Bench provide a structured and extensible foundation for measuring and advancing the formal-proof capabilities of AI systems in quantum science. We expect these benchmarks to support the development of more capable and reliable proof agents. Moreover, they can guide the improvement of AI systems through fine-tuning, post-training, and harness-system design, paving the way toward self-evolving AI scientists for advancing quantum information science.

Acknowledgment

This work was partially supported by the National Natural Science Foundation of China (Grant Nos. 92576114, 12447107) and the Guangdong Provincial Quantum Science Strategic Initiative (Grant Nos. GDZX2403008, GDZX2503001, and GDZX2403001).

References

[1] Peter W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994.

[2] Göran Lindblad. Completely positive maps and entropy inequalities. *Communications in Mathematical Physics*, 40(2):147–151, 1975.

[3] Mark M. Wilde. *Quantum Information Theory*. Cambridge University Press, 2 edition, 2017.

[4] Anthony Bordg, Hanna Lachnitt, and Yijun He. Certified quantum computation in Isabelle/HOL. *Journal of Automated Reasoning*, 65(5):691–709, 2021.

[5] Yuxiang Peng, Kesha Hietala, Runzhou Tao, Liyi Li, Robert Rand, Michael Hicks, and Xiaodi Wu. A formally certified end-to-end implementation of Shor's factorization algorithm. *Proceedings of the National Academy of Sciences*, 120(21):e2218775120, 2023.

[6] Alex Meiburg, Leonardo A. Lessa, and Rodolfo R. Soldati. A formalization of the generalized quantum Stein's lemma in Lean, 2025.

[7] Kazumi Kasaura, Kei Tsukamoto, Kento Mori, Risa Mizuno, Takahiro Namatame, Yuta Oriike, Masaya Taniguchi, Sho Sonoda, and Hayata Yamasaki. Lean-Quantum: Toward AI-assisted formalization of quantum information, 2026.

[8] Chengkai Zhu, Ziao Tang, Guocheng Zhen, Yimeng Cao, Yusheng Zhao, Ranyiliu Chen, Xuanqiang Zhao, Lei Zhang, and Xin Wang. Lean-QIT: Towards a formal infrastructure for quantum information theory, July 2026.

[9] Lei Zhang, Yusheng Zhao, Hongshun Yao, and Xin Wang. Building Shor's algorithm in Lean: An agentic formalization of quantum attacks on RSA-2048 and P-256, July 2026.

[10] Mingrui Jing, Lei Zhang, Yusheng Zhao, Hongshun Yao, and Xin Wang. An agentic formalization for certified quantum neural network design, 2026.

[11] Mattias Ehatamm, Yi Lee, Xiaodi Wu, and Runzhou Tao. End-to-end formalization of quantum error correction, 2026.

[12] Uri Kol, Maor Ben-Shahar, Kfir Sulimany, and Dirk Englund. A machine-verified proof of a quantum-optimization conjecture, 2026.

[13] Amitayush Thakur, George Tsoukalas, Yeming Wen, Jimmy Xin, and Swarat Chaudhuri. An in-context learning agent for formal theorem-proving. In *First Conference on Language Modeling*, 2024.

[14] Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, Ottavia Bertolli, Tom Zahavy, Amol Mandhane, Jessica Yung, Iuliya Beloshapka, Borja Ibarz, Vivek Veeriah, Lei Yu, Oliver Nash, Paul Lezeau, Salvatore Mercuri, Calle Sønne, Bhavik Mehta, Alex Davies, Daniel Zheng, Fabian Pedregosa, Yin Li, Ingrid von Glehn, Mark Rowland, Samuel Albanie, Ameya Velingker, Simon Schmitt, Edward Lockhart, Edward Hughes, Henryk Michalewski, Nicolas Sonnerat, Demis Hassabis, Pushmeet Kohli, and David Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2026.

[15] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, Wenjun Gao, Qihao Zhu, Dejian Yang, Zhibin Gou, Z. F. Wu, Fuli Luo, and Chong Ruan. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search, 2024.

[16] Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition, 2025.

[17] Junqi Liu, Zihao Zhou, Zekai Zhu, Marco Dos Santos, Weikun He, Jiawei Liu, Ran Wang, Yunzhou Xie, Junqiao Zhao, Qiufeng Wang, Lihong Zhi, Jia Li, and Wenda Li. Numina-Lean-Agent: An open and general agentic reasoning system for formal mathematics, 2026.

[18] Borja Requena, Austin Letson, Krystian Nowakowski, Izan Beltran-Ferreiro, and Leopoldo Sarra. A minimal agent for automated theorem proving, 2026.

[19] Jui-Hui Chung, Ziyang Cai, Zihao Li, Qishuo Yin, Rohit Agarwal, Simon Park, Rodrigo Porto, Narutatsu Ri, Ziran Yang, Shange Tang, Xingyu Dang, Hongzhou Lin, Mengdi Wang, Danqi Chen, Chi Jin, Liam H. Fowl, and Sanjeev Arora. Goedel-Architect: Streamlining formal theorem proving with blueprint generation and refinement, 2026.

[20] Haocheng Ju, Guoxiong Gao, Jiedong Jiang, Bin Wu, Zeming Sun, Shurui Liu, Leheng Chen, Yutong Wang, Yuefeng Wang, Zichen Wang, Wanyi He, Peihao Wu, Liang Xiao, Ruochuan Liu, Bryan Dai, and Bin Dong. Automated conjecture resolution with formal verification, 2026.

[21] Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. LeanSearch v2: Global premise retrieval for lean 4 theorem proving, 2026.

[22] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations*, 2022.

[23] George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. In *Advances in Neural Information Processing Systems*, volume 37. Curran Associates, Inc., 2024.

[24] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Animashree Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc., 2023.

[25] Jiewen Hu, Thomas Zhu, and Sean Welleck. miniCTX: Neural theorem proving with (long-)contexts. In *The Thirteenth International Conference on Learning Representations*, 2025.

[26] Azim Ospanov, Farzan Farnia, and Roozbeh Mohit. miniF2F-Lean Revisited: Reviewing limitations and charting a path forward. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025.

[27] Pawan Sasanka Ammanamanchi, Siddharth Bhat, and Stella Biderman. Faults in our formal benchmarking: Dataset defects and evaluation failures in Lean theorem proving, 2026. Accepted at ICML 2026.

[28] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In *Automated Deduction — CADE-25*, pages 378–388. Springer, 2015.

[29] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE 28*, pages 625–635. Springer, 2021.

[30] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

[31] Andrej Bauer, Matej Petković, and Ljupco Todorovski. MLFMF: Data sets for machine learning for mathematical formalization. In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc., 2023.

[32] Huaiyuan Ying, Zijian Wu, Yihan Geng, Jiayu Wang, Dahua Lin, and Kai Chen. Lean Workbook: A large-scale Lean problem set formalized from natural language math problems. In *Advances in Neural Information Processing Systems*, volume 37. Curran Associates, Inc., 2024.

[33] Auguste Poiroux, Antoine Bosselut, and Viktor Kunčak. RLMEval: Evaluating research-level neural theorem proving. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10946–10957, Suzhou, China, November 2025. Association for Computational Linguistics.

[34] Haoyu Zhao, Yihan Geng, Shange Tang, Yong Lin, Bohan Lyu, Hongzhou Lin, Chi Jin, and Sanjeev Arora. Ineq-Comp: Benchmarking human-intuitive compositional reasoning in automated theorem proving of inequalities. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025.

[35] Rui Yang, Ziruo Wang, Yuntian Gu, Yitao Liang, and Tongyang Li. QCircuitBench: A large-scale dataset for benchmarking quantum algorithm design. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025.

[36] Wentao Long, Yunfei Zhang, Chenyi Li, Li Zhou, Chumin Sun, and Zaiwen Wen. CAM-Bench: A benchmark for computational and applied mathematics in Lean, 2026.

[37] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.

[38] Marco Tomamichel. *Quantum Information Processing with Finite Resources: Mathematical Foundations*, volume 5 of *SpringerBriefs in Mathematical Physics*. Springer International Publishing, 2016.

[39] Masahito Hayashi and Hiroshi Nagaoka. General formulas for capacity of classical-quantum channels. *IEEE Transactions on Information Theory*, 49(7):1753–1768, 2003.

[40] Seth Lloyd. Capacity of the noisy quantum channel. *Physical Review A*, 55(3):1613–1622, 1997.

[41] Benjamin Schumacher. Quantum coding. *Physical Review A*, 51(4):2738–2747, 1995.

[42] Alexander S. Holevo. The capacity of the quantum channel with general signal states. *IEEE Transactions on Information Theory*, 44(1):269–273, 1998.

[43] Charles H. Bennett, Gilles Brassard, Sandu Popescu, Benjamin Schumacher, John A. Smolin, and William K. Wootters. Purification of noisy entanglement and faithful teleportation via noisy channels. *Physical Review Letters*, 76(5):722–725, 1996.

[44] Robert Alicki and Mark Fannes. Continuity of quantum conditional information. *Journal of Physics A: Mathematical and General*, 37(5):L55–L57, 2004.

[45] Andreas Winter. Tight uniform continuity bounds for quantum entropies: conditional entropy, relative entropy distance and energy constraints. *Communications in Mathematical Physics*, 347(1):291–313, 2016.

[46] Ethan Bernstein and Umesh Vazirani. Quantum complexity theory. *SIAM Journal on Computing*, 26(5):1411–1473, 1997.

[47] Andrew M. Childs and Nathan Wiebe. Hamiltonian simulation using linear combinations of unitary operations. *Quantum Information and Computation*, 12(11–12):901–924, 2012.

[48] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2019, pages 193–204, New York, NY, USA, 2019. Association for Computing Machinery.

[49] Guang Hao Low and Isaac L. Chuang. Optimal Hamiltonian simulation by quantum signal processing. *Physical Review Letters*, 118(1):010501, 2017.

[50] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018.

[51] A. Yu. Kitaev. Quantum measurements and the Abelian Stabilizer Problem, 1995.

[52] Dominic W. Berry, Andrew M. Childs, Richard Cleve, Robin Kothari, and Rolando D. Somma. Simulating Hamiltonian dynamics with a truncated Taylor series. *Physical Review Letters*, 114(9):090502, 2015.

[53] Charles H. Bennett, Herbert J. Bernstein, Sandu Popescu, and Benjamin Schumacher. Concentrating partial entanglement by local operations. *Physical Review A*, 53(4):2046–2052, 1996.

[54] Christopher A. Fuchs and Jeroen van de Graaf. Cryptographic distinguishability measures for quantum-mechanical states. *IEEE Transactions on Information Theory*, 45(4):1216–1227, 1999.

[55] Eric Chitambar, Debbie Leung, Laura Mančinska, Maris Ozols, and Andreas Winter. Everything you always wanted to know about LOCC (but were afraid to ask). *Communications in Mathematical Physics*, 328(1):303–326, 2014.

Table 4: QAlg-Bench task inventory.

Field	Title	Difficulty
SOM	Spectral projectors of a Boolean phase oracle	6
	Orthonormality of coherent Markov-chain rows	5
	Unitary extension of a Markov-chain isometry	9
	Post-measurement state after observing one qubit	2
	Linear dependence over R versus over F2	6
	Fidelity of a Hamming-weight-truncated phase state	10
	Weyl-average sanity check for the shift operator	10
	Hilbert–Schmidt invariance under transposition	4
	Trivial common unitary stabilizer of two full-rank states	10
	Local marginal after an unread projective measurement	5
CAA	Unitarity and Hermiticity of the bit oracle	2
	Matrix of H tensor H	3
	Controlled phase from CNOT and Hadamards	6
	Walsh–Hadamard decoding of a Boolean character	9
	Controlled unitary invariant subspaces	3
	Vanishing Pauli-Y expectation for real circuits	7
	Exact QROM factorization into commuting address-selective operators	9
	Phase kickback for a controlled eigenunitary	4
	Two-CNOT synthesis of a controlled Ry rotation	9
	Unitarity of the finite quantum Fourier transform	8
	Product-state factorization of $\mathrm{QFT}_8\|3\rangle$	6
	QPE with a superposition of exact eigenvectors	7
	Eigenvectors of modular multiplication in Shor's algorithm	10
	Weak Fourier sampling for a transposition in S_n	9
	Fidelity bound for hidden subgroup coset states	3
	Rank of the strong Fourier sampling conditional state	10
SSL	Leading commutator term of a two-term exponential product	9
	First-order Lie–Trotter global error scaling	9
	Accumulation of product-formula errors for many small terms	9
	Coefficient 1-norm in truncated Taylor simulation	8
	LCU block encoding from PREPARE and SELECT	9
	Postselected spectral transformation by trigonometric QSP	10
	Shared-parameter shift rule for $R_X \otimes R_Z$	10
	Third moment from two controlled-SWAP tests	9
	Parity and maximum of a Fejer-kernel QSP response	10
	Approximate two-design depth and Haar loss fluctuations	10

A Benchmark settings

The following inventories list the 36 QAlg-Bench tasks and 40 QIT-Bench tasks used in this study. Each row records its field assignment, title, and difficulty. Difficulty is a model-assisted integer assignment from 1 to 10 that was frozen before model execution. Two separate rubric reviews score proof structure, theory dependencies, Lean engineering, and mathematical technique on four axes scored from 1 to 4; the combined rubric maps these assessments to the 1–10 scale, and a difference greater than one point receives adjudication. The resulting value is used only as the weight d_i in Equation (1); it is not an intrinsic measure of proof complexity and is not inferred from model performance. The tables are generated deterministically from the fixed benchmark inventory, whose difficulty values agree with the evaluation data used in this paper.

For each model in Table 6, baseline and LAD use the same model identifier, agent tool, and reasoning setting. Each recorded task receives one attempt with a 30-minute model invocation limit, and tasks run serially in fresh isolated workspaces. The agent may edit only the theorem body and may run Lean on the current target, receiving the resulting Lean output. No partial conversation or answer is resumed, and external network access is disabled. If an infrastructure interruption requires replacement, the replacement starts with a fresh model invocation. After every invocation, including a timeout, an independent Lean verification checks the final submission. The model and agent tool columns identify the evaluated settings. In Context window, an em dash marks runs without a separately pinned long-context setting.

Table 5: QIT-Bench task inventory.

Field	Title	Difficulty
QCR	Unitarity of a System–Environment Evolution	6
	Kraus Representation from a System–Environment Unitary	7
	The Partial Trace as a Completely Positive Trace-Preserving Map	6
	Trace Preservation in the Choi Representation	5
	Unitality in the Choi Representation	4
	Choi Matrix Acting on a Canonical Input State	8
	Choi Positivity Characterization of Complete Positivity	10
	A Fixed Point of Every Trace-Preserving Quantum Operation	10
	Primitivity and Nondegeneracy of the Stationary Eigenvalue	10
	A Partial-SWAP Marginal Splitter	8
	Hadamard Channel from a Dephasing Isometry	6
GSD	The Choi Matrix of the Qutrit Werner–Holevo Channel	7
	Trace Preservation and Unitality of the Werner–Holevo Channel	4
	No Maximally Entangled Vector in the Qutrit Antisymmetric Subspace	5
	The Symmetric Projector	9
	The Antisymmetric Projector	9
	Dimension of the Symmetric Subspace for Three Copies	9
	Dimension of the Antisymmetric Subspace for Three Copies	10
	Trace Norm and Unitary Optimization	10
	Variational Characterization of the Trace Norm	10
	Maximally Entangled State and the Hilbert–Schmidt Inner Product	1
	Trace Distance Lower Bound for a Pure State	10
	Trace-Distance Data Processing for Quantum Channels	10
	Fuchs–van de Graaf Inequalities	10
	Gentle Measurement Lemma for the Normalized Post-Measurement State	10
	Alberti's Theorem in the Commuting Case	10
	A Universal Upper Bound from Alberti's Theorem	7
	Channel Fidelity and Gate Fidelity for a Pure Input State	7
IME	Negative Conditional Entropy and Entanglement for Pure Bipartite States	10
	Random Unitary Realization of the Completely Depolarizing Channel	10
	Uniqueness of the Maximum-Entropy State	10
	Projective Measurement as a Random Unitary Channel	6
	Entropy Increase under Non-Selective Projective Measurement	8
	Convexity of Quantum Mutual Information	9
	Holevo Bound and the Classical Capacity of n Qubits	10
	Quantum Fano Inequality	10
	Spectrum and Entropy of a Single-Qubit Source	8
	Exact Entanglement Dilution to a Maximally Entangled State	8
	Achievability of Entanglement Concentration	10
	Converse for Entanglement Concentration	10

Table 6: Model and agent settings used for evaluation.

Model	Developer	Model identifier	Agent tool	Reasoning setting	Context window
GPT-5.5	OpenAI	`gpt-5.5`	Codex	xhigh	256k
Kimi K3	Moonshot AI	`k3[1m]`	Claude Code	max	1M
DeepSeek V4-Pro	DeepSeek	`deepseek-v4-pro[1m]`	Claude Code	max	1M
MiniMax M3	MiniMax	MiniMax M3	Claude Code	max	—

B More experimental details

This appendix provides the absolute resource metrics, within-suite rankings, and field values underlying the main evaluation.

Table 7 reports completion and the per-score cost values behind Table 1.

Table 7: Absolute completion and per-score cost metrics. Completion is the verified-problem percentage, and economic and time cost are reported in USD and seconds per score point, respectively. Economic cost divides mean API list-price-equivalent USD among tasks with reconstructable prices by the corresponding difficulty-weighted score. Time cost is the full-suite mean model invocation seconds per task divided by the overall score; model invocation time includes the complete agent invocation and Lean checks initiated by the agent but excludes the independent final verification. A black em dash denotes an unavailable observed metric.

Suite	Condition	Model	Completion	USD/score point	s/score point
QAlg-Bench	baseline	GPT-5.5	61.1%	0.040530	9.607
		Kimi K3	47.2%	0.019408	34.806
		DeepSeek V4-Pro	41.7%	0.001098	48.629
		MiniMax M3	8.3%	—	118.860
	LAD	GPT-5.5	69.4%	0.033640	7.729
		Kimi K3	50.0%	0.023276	32.678
		DeepSeek V4-Pro	52.8%	0.000977	32.528
		MiniMax M3	25.0%	—	89.355
QIT-Bench	baseline	GPT-5.5	52.5%	0.049337	19.287
		Kimi K3	27.5%	0.046758	74.849
		DeepSeek V4-Pro	22.5%	0.004859	88.594
		MiniMax M3	12.5%	—	172.318
	LAD	GPT-5.5	65.0%	0.042586	12.949
		Kimi K3	30.0%	0.045455	70.350
		DeepSeek V4-Pro	25.0%	0.002895	85.048
		MiniMax M3	15.0%	—	135.409

Figure 4 presents the absolute scores as a compact within-suite ranking. Each model has adjacent baseline and LAD bars, and the printed values provide the exact comparison.

Table 8 gives the exact field values behind Figure 3. The QAlg-Bench and QIT-Bench subtables spell out baseline and LAD for each model. Their fixed difficulty denominators are 67, 105, and 93 for SOM, CAA, and SSL, and 80, 138, and 109 for QCR, GSD, and IME. Within each suite, the three field partitions are disjoint and cover the complete task set.

Together, these values provide a numerical basis for checking the overall and field-level comparisons in Section 4.

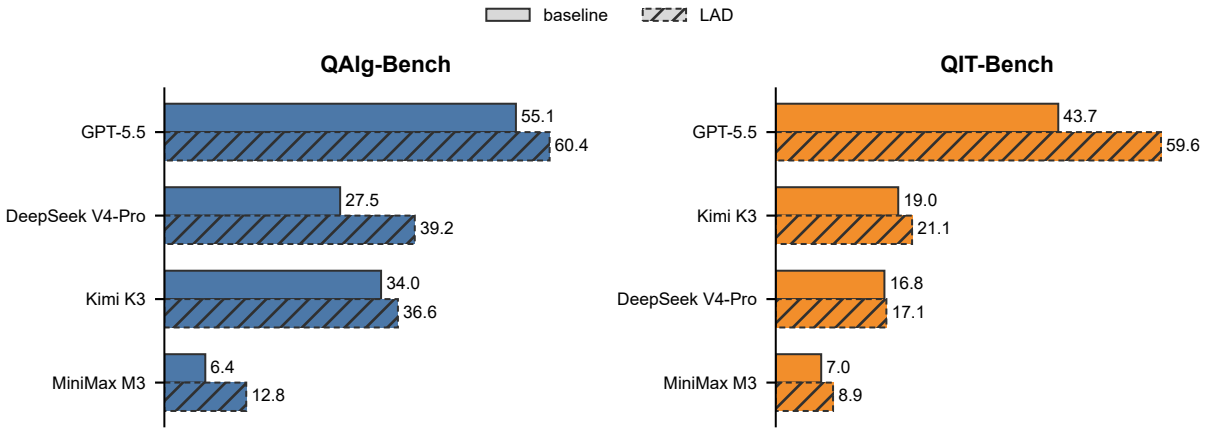

Figure 4: Within-suite score leaderboards for QAlg-Bench and QIT-Bench. Each model has adjacent baseline and LAD bars, with exact difficulty-weighted scores printed at the bar ends. Models are ordered within each suite by the larger of their two condition scores; ties are ordered as GPT-5.5, Kimi K3, DeepSeek V4-Pro, and MiniMax M3. Blue and orange identify QAlg-Bench and QIT-Bench, while solid outlines identify baseline and dashed, diagonally hatched outlines identify LAD.

Table 8: Difficulty-normalized field scores by suite and condition. Subtables (a,b) report QAlg-Bench and QIT-Bench, respectively. The fixed difficulty denominators are SOM=67, CAA=105, SSL=93 for QAlg-Bench and QCR=80, GSD=138, IME=109 for QIT-Bench.

(a) QAlg-Bench

Condition	Model	SOM	CAA	SSL
baseline	GPT-5.5	58.2	76.2	29.0
	Kimi K3	41.8	41.9	19.4
	DeepSeek V4-Pro	41.8	33.3	10.8
	MiniMax M3	7.5	11.4	0.0
LAD	GPT-5.5	70.1	81.9	29.0
	Kimi K3	41.8	48.6	19.4
	DeepSeek V4-Pro	41.8	46.7	29.0
	MiniMax M3	17.9	21.0	0.0

(b) QIT-Bench

Condition	Model	QCR	GSD	IME
baseline	GPT-5.5	75.0	50.0	12.8
	Kimi K3	26.2	25.4	5.5
	DeepSeek V4-Pro	33.8	13.0	9.2
	MiniMax M3	11.2	10.1	0.0
LAD	GPT-5.5	87.5	80.4	12.8
	Kimi K3	35.0	25.4	5.5
	DeepSeek V4-Pro	26.2	13.8	14.7
	MiniMax M3	18.8	10.1	0.0